1. Interface

Why not just use `wire`/`reg` in the testbench?*

The Problem without Interfaces:

- If you have 50 signals, you'd pass 50 arguments to every module/task

- If you change one signal (add a parity bit), you modify every single module

- Direction (`input`/`output`) confusion between DUT and testbench

Why Interfaces?

Encapsulation: Like a USB cable - one connector carries power, data, ground. The interface bundles all signals into one handle.

Reusability: Change from APB to AHB protocol? Just swap the interface, keep the testbench structure.

Clean Hierarchy: Top-level connects interface once, lower levels just receive the interface handle.

Exam Tip: Mention "**signal bundle abstraction"** and "**reduction of top-level connectivity errors**."

---

2. Transaction Class

Why create a class just to hold data?

The Problem without Transactions:

- Driving signals directly: `drv.a = 8'hFF; drv.b = 8'h00;`

- You lose the **temporal information** (when this happened)

- Cannot randomize or constrain scalar values easily

Why Transaction Objects:

Abstraction Level: Think of it as a "packet" - in networking, you don't send individual bits; you send frames. Similarly, don't toggle signals; send transactions.

Randomization: SystemVerilog can randomize objects (`tran.randomize()`) but not loose variables efficiently.

Queue/Storage: You can put objects in mailboxes/queues. You cannot put loose signals in a mailbox.

Copying: `tran.copy()` creates history/logs without reference issues.

Memory Trick: Transaction = Letter in an envelope (data + metadata), while signals = Raw ink on paper.

---

3. Mailbox

Why not use global variables or events to pass data?

The Problem without Mailboxes:

// BAD approach

transaction t;

gen.t = new(); // Generator creates

drv.t = gen.t; // Driver reads - RACE CONDITION!

// Did generator finish writing before driver read?

```

**Why Mailboxes:**

**Synchronization**: `put()` blocks if full, `get()` blocks if empty. It's a **handshake**.

**Thread Safety**: Generator and Driver run in parallel (`fork-join`). Mailbox is the safe "mailbox slot" between them - like a bank deposit box.

**Buffering**: Generator can create 5 transactions ahead while Driver processes 1. Mailbox acts as FIFO.

Key Methods for Exam:

- `put()` = Deposit letter (wait if box full)

- `get()` = Retrieve letter (wait if box empty)

- `peek()` = Look at letter without removing it (used in scoreboards sometimes)

---
```

4. Clocking Blocks

Why not just use `always @(posedge clk)` everywhere?

The Problem:

// Race condition nightmare

always @(posedge clk) a <= 1; // DUT drives

always @(posedge clk) b = a;   // TB captures - Did it get old or new value?

Why Clocking Blocks:

Timing Discipline: `clocking cb @(posedge clk)` creates a **sampling moment** (usually 1ns before posedge or at posedge with delta cycles).

Separation of Concerns:

Testbench uses `cb` (synchronous, sampled)

DUT connects to raw signals (asynchronous to testbench timing)

Skew Control: `clocking cb @(posedge clk) default input #1 output #0;` means sample inputs 1ns before clock, drive outputs at clock edge.

Exam Answer: *"Clocking blocks eliminate race conditions by providing a single, predictable sampling point for synchronous signals."*

---

5. Virtual Interface

Why `virtual interface` and not just `interface`?

The Problem:

- Classes are instantiated at simulation time (dynamic).

- Physical interfaces exist at elaboration time (static).

- You cannot put a physical interface inside a class that gets created/destroyed dynamically.

Why Virtual:

Indirection: `virtual` means "pointer to the actual interface." Like having a phone number (virtual) vs. the actual house (physical interface).

Passing to Classes: Driver/Monitor are classes. They need a handle (reference) to the physical interface at the top level.

Multiple Instances: One Driver class code can drive Interface_0 or Interface_1 via virtual handle.

Exam Tip: Virtual interface = **Remote control** for the TV (interface). Multiple remotes can control one TV, or one remote can control different TVs.

---

6. Generator + Driver Separation

Why not have Generator drive signals directly?

Single Responsibility Principle:

Generator: Knows **what** to test (scenarios, constraints, sequences)

Driver: Knows **how** to wiggle pins (timing, protocols, bus cycles)

Why Separate:

Reuse: Same Generator can feed a Driver for APB, AXI, or SPI - just swap the Driver.

Abstraction Levels: Generator works at transaction level ("Send write command"), Driver works at signal level ("Assert AWVALID, put address on bus, wait for AWREADY").

Synchronization: Generator creates transactions at random speed; Driver consumes at bus speed. Mailbox decouples their timing.

Analogy: Generator = Chef (prepares recipe), Driver = Waiter (delivers to table). Chef shouldn't also serve tables - different skillsets and speeds.

---

7. Monitor (Separate from Driver)

Why not reuse Driver code to observe?

The Problem:

- Driver is focused on driving - it doesn't care about response timing.

- If DUT delays output by 5 cycles, Driver is already moving to next transaction.

Self-checking requires observing actual DUT outputs, not just expected inputs.

Why Separate Monitor:

Passive Observation: Monitor never drives signals (avoids contention). It's a "fly on the wall."

Coverage Collection: Monitors feed functional coverage (did we see error response? did we see burst mode?)

Protocol Checking: Monitor checks if DUT follows protocol (e.g., "Valid cannot be high for more than 16 cycles").

Exam Answer: "Monitor observes the DUT's output interface to reconstruct the transaction that actually occurred, independent of what was expected."

---

8. Scoreboard

Why not just use `$display` in the monitor?

The Problem:

Checking inside Monitor mixes observation with validation. What if you need to compare against a C model later? Or against reference memory?

Why Scoreboard Architecture:

Reference Model: Scoreboard contains a "golden" model (often high-level C/SystemC or abstract SV code) that predicts correct behavior.

Comparison: Actual (from Monitor) vs. Expected (from Predictor/Reference).

Ordering: Scoreboards can handle out-of-order responses (AXI, USB) using queues to match transactions.

Types for Exam:

1. In-order: FIFO comparison (simple)

2. Out-of-order: Sort by ID/Tag then compare (complex buses)

3. Statistical: Check only final state, not every cycle

---

9. Environment Class

Why wrap everything in another class?

Why Encapsulate:

Test Independence: Each test creates an Environment with specific configurations (e.g., `env.cfg.error_injection = 1`).

Phasing: Environment controls the simulation phases:

1. Build: Create objects
2. Connect: Connect mailboxes
3. Run: Execute test
4. Report: Print results

Reusability: Same Environment used in Test 1 (basic) and Test 2 (stress) - just change Generator's transaction count.

Exam Concept: This mirrors UVM's `uvm_env` - showing you understand structured verification methodology.

---

10. Events (`->` and `@`)

When to use Events vs Mailboxes?

Events = Doorbell (notification only, no data)

Mailboxes = Mailbox (notification + data payload)

Use Events when:

- Driver tells Generator "I'm done with this transaction" (`->drv_done`)

- Synchronization without data transfer (barrier synchronization)

Use Mailboxes when:

- Passing the actual transaction object between components

-

Component	Analogy	Key Purpose
Interface	USB Cable	Bundle signals
Transaction	Envelope	Data container
Mailbox	Bank deposit box	Safe data exchange between threads
Generator	Chef	Creates test scenarios
Driver	Waiter	Converts transactions to signals
Monitor	Security camera	Observes without interfering
Scoreboard	Judge	Compares actual vs expected
Clocking Block	Metronome	Defines exact sampling timing
Virtual Interface	Phone number	Remote control of physical signals